

Project Report

Executive summary, architecture, milestones, results and conclusion

FitBuddy — Project Report

AI-Powered Personal Fitness Coach

Executive Summary

FitBuddy is a full-stack web application that harnesses the power of **Google Gemini 2.0 Flash AI** to deliver personalized fitness coaching at scale. The application eliminates the need for expensive personal trainers by providing:

- **Instant personalized 7-day workout plans** tailored to each user's age, weight, goal, and intensity level
- ▬ **Real-time plan adaptation** via natural language feedback
- **AI-generated nutrition and recovery guidance** aligned with fitness goals

Built with **FastAPI (Python)** on the backend and **Vanilla HTML/CSS/JavaScript** on the frontend, FitBuddy demonstrates how generative AI can transform personal health and fitness accessibility.

1. Project Title & Description

Field	Details
Project Name	FitBuddy — AI-Powered Personal Fitness Coach
Domain	Health & Fitness / Artificial Intelligence
Technology	FastAPI, Google Gemini AI, SQLite, HTML/CSS/JS

AI Model	Google Gemini 2.0 Flash
Type	Full-Stack Web Application

Description

FitBuddy is a conversational AI fitness platform that collects user health data and uses Google's state-of-the-art Gemini language model to generate unique, structured, day-by-day workout routines. The platform supports three distinct AI-powered scenarios — plan generation, plan refinement via feedback, and nutritional guidance — stored persistently in a relational database for long-term user history tracking.

2. Use Cases

Use Case 1: Personalized Workout Plan Generation

Actor: New user

Goal: Receive a tailored 7-day workout plan

Flow:

1. User opens FitBuddy and fills in their profile (Name, Age, Weight, Goal, Intensity)
2. User clicks "Generate My 7-Day Plan"
3. System sends profile to Gemini AI via structured prompt
4. AI generates a complete week-long plan in JSON format
5. System displays plan in interactive day cards
6. User receives a unique User ID for future sessions

Outcome: User has a personalized, actionable 7-day fitness plan in under 20 seconds.

Use Case 2: Plan Refinement via Natural Language Feedback

Actor: Returning user with existing plan

Goal: Modify the plan based on personal preferences or physical feedback

Flow:

1. User navigates to "Update Plan" tab
2. User enters their User ID and describes desired changes
3. User can use pre-built feedback chips or type custom feedback
4. System sends profile + feedback to Gemini AI
5. AI generates a refined plan incorporating the feedback
6. Updated plan is displayed with "Feedback Applied" confirmation

Sample Feedback: *"I want more focus on cardio and fewer rest days. Also add more core exercises."*

Outcome: Plan is dynamically updated to match user's evolving needs.

Use Case 3: Nutrition & Recovery Guidance

Actor: Any user (new or returning)

Goal: Get dietary and recovery advice for their specific fitness goal

Flow:

- 1. User navigates to "Nutrition Tips" tab
- 2. User selects their fitness goal from dropdown
- 3. Optionally provides User ID for personalized name
- 4. System queries Gemini AI with a nutrition-focused prompt
- 5. AI returns structured nutrition advice (foods, hydration, recovery)
- 6. Results displayed in a structured card format

Outcome: User receives actionable, goal-specific dietary guidance.

3. Technical Architecture

System Architecture Diagram



Data Flow Diagram



4. Implementation Details

4.1 Backend (FastAPI + Python)

File: `backend/main.py` (277 lines)

Key Implementation Choices:

- **FastAPI** chosen for async support, automatic OpenAPI docs, and Pydantic integration
- **SQLite** chosen for zero-configuration persistence, suitable for demo scale
- **Pydantic v2** models ensure input validation before any AI call is made
- **Row factory** (`conn.row_factory = sqlite3.Row`) enables dict-style row access
- **Structured JSON prompts** with explicit schema in system message ensure reliable parsing

Database Initialization:

```
def init_db(): conn = sqlite3.connect(DB_PATH) # Creates users, workout_plans, nutrition_tips tables
conn.commit() conn.close() init_db() # Called at server startup
```

4.2 AI Integration (Google Gemini)

Prompt Engineering Strategy: Role-based system prompts with explicit JSON schema

```
Role → "You are FitBuddy, an expert personal fitness coach..." Context → User profile (name, age,
weight, goal, intensity) Constraints → "Return ONLY valid JSON in this exact format..." Schema →
Complete JSON structure with all required fields
```

Reliability Feature — Code Fence Stripping:

```
if text.startswith("```"): text = text.split("```")[1] if text.startswith("json"): text = text[4:]
```

This handles the common case where Gemini wraps responses in markdown code blocks.

4.3 Frontend Design System

File: `frontend/static/style.css` (880 lines)

Glassmorphism Design: Cards use `backdrop-filter: blur(12px)` with semi-transparent backgrounds.

Animation System: 8 custom keyframe animations for micro-interactions:

- `orbFloat` — background depth
- `fadeSlideDown` — entrance animations
- `slideInLeft/Right` — notifications
- `pulse` — live indicator

- `spin` — loading states

Component Architecture (in pure CSS):

- Design tokens via CSS custom properties (`--accent-primary`, `--grad-green`, etc.)
 - Component classes: `.card`, `.btn-primary`, `.day-card`, `.exercise-item`
 - State classes: `.active`, `.selected`, `.expanded`
-

5. Milestones

Milestone 1: Ideation & Design (Week 1)

- ■ Defined problem statement and target user persona
- ■ Created empathy map and brainstorming documents
- ■ Identified core feature set (3 AI scenarios)
- ■ Selected technology stack (FastAPI + Gemini + SQLite)
- ■ Designed UI wireframes and component hierarchy

Milestone 2: Backend Development (Week 2)

- ■ Set up FastAPI project with CORS middleware
- ■ Designed SQLite database schema (3 tables)
- ■ Implemented `init_db()` with auto-creation
- ■ Built Pydantic models for request validation
- ■ Integrated Google Gemini SDK
- ■ Implemented all 5 API routes
- ■ Added error handling for AI parse failures

Milestone 3: AI/ML Integration & Prompt Engineering (Week 2-3)

- ■ Designed role-based system prompts for Scenario 1 (workout generation)
- ■ Extended prompt with feedback injection for Scenario 2 (plan update)
- ■ Created separate nutritionist persona prompt for Scenario 3 (nutrition)
- ■ Implemented JSON response parsing with code fence stripping
- ■ Tested and tuned prompts for consistent JSON output

Milestone 4: Frontend Development (Week 3)

- Built glassmorphism dark-mode design system
- Implemented animated background orbs and hero section
- Built 3-tab navigation with animated panel switching
- Created interactive goal and intensity card selectors
- Implemented loading states with progress bars
- Built dynamic day card renderer from JSON plan data
- Implemented toast notification system
- Built nutrition results card renderer
- Added counter animations for hero statistics

Milestone 5: Testing & Documentation (Week 4)

- Unit testing for all API endpoints
- End-to-end integration testing for all 3 scenarios
- UI/UX testing across browsers and screen sizes
- Performance measurement (latency benchmarks)
- Edge case and error handling tests
- Documentation created for all phases
- README updated with setup instructions

6. Results

Quantitative Results

Metric	Value
API Endpoints Implemented	5
AI Scenarios Covered	3
Average Plan Generation Time	~13 seconds

Average Nutrition Tip Time	~6 seconds
Frontend Components	20+
Lines of Code (Backend)	277
Lines of Code (Frontend JS)	478
Lines of Code (CSS)	880
Database Tables	3
Unit Tests Passed	18/18

Qualitative Results

1. **Personalization:** Every plan generated is unique to the user's profile — different exercises, durations, motivational notes
2. **Adaptability:** The feedback system successfully modifies plans based on casual natural language input
3. **Visual Impact:** The glassmorphism dark UI with animated elements creates a premium, modern aesthetic
4. **Reliability:** JSON parsing handles both plain and markdown-wrapped AI responses
5. **Completeness:** All three required scenarios are fully implemented and functional

7. Conclusion

FitBuddy successfully demonstrates the potential of **generative AI in the health and fitness domain**. By combining FastAPI's high-performance async backend with Google Gemini's language generation capabilities, the project achieves:

- **Democratized fitness coaching** — AI replaces expensive personal trainers
- **True personalization** — Each plan is uniquely generated for each user
- **Dynamic adaptability** — Plans evolve with user feedback, preventing drop-off
- **Practical nutrition guidance** — Completes the fitness-nutrition loop

The project explores three key AI applications:

1. **Generative AI for structured content** (workout plans as JSON)

2. **Context-aware AI updates** (incorporating user feedback into regeneration)
3. **Domain-specific AI personas** (nutritionist role vs. fitness coach role)

Key Learnings

- Prompt engineering is critical — the quality of the JSON schema in the prompt directly impacts AI output reliability
- Code fence stripping is essential for production-ready LLM integrations
- FastAPI + SQLite + Gemini is a powerful and lightweight full-stack AI pattern
- Glassmorphism with animated elements significantly enhances user engagement

Future Enhancements

1. **Streaming responses** — Show plan generating in real-time
2. **User authentication** — Secure user accounts with JWT
3. **Progress tracking** — Weekly check-ins and plan progression
4. **PostgreSQL migration** — Production-grade database
5. **Mobile app** — React Native wrapper for native mobile experience
6. **Exercise videos** — YouTube API integration for exercise demonstrations
7. **Wearable integration** — Connect with Fitbit/Apple Watch data

8. References

1. Google Gemini API Documentation — <https://ai.google.dev/docs>
 2. FastAPI Documentation — <https://fastapi.tiangolo.com>
 3. Uvicorn ASGI Server — <https://www.uvicorn.org>
 4. SQLite Documentation — <https://sqlite.org/docs.html>
 5. Google AI Studio (API Key) — <https://aistudio.google.com/apikey>
 6. Pydantic v2 Documentation — <https://docs.pydantic.dev>
 7. MDN Web Docs (CSS Animations) — <https://developer.mozilla.org/en-US/docs/Web/CSS/animation>
 8. Google Fonts — <https://fonts.google.com>
-

Appendix A: API Documentation

See `backend/main.py` or run the server and visit `http://localhost:8000/docs` for the interactive OpenAPI documentation.

Appendix B: Setup Instructions

See `README.md` in the project root for step-by-step setup and running instructions.

Appendix C: GitHub Repository Structure

```
FitBuddy/
├── README.md # Project overview and setup
├── start.bat # Windows quick-start script
├── 1_Ideation_Phase/
│   ├── Problem_Statement.md
│   ├── Brainstorming.md
│   └── Empathy_Map.md
├── 2_Requirement_Analysis/
│   ├── Requirement_Analysis.md
├── 3_Frontend_Development/
│   ├── Frontend_Documentation.md
├── 4_Backend_Development/
│   ├── Backend_Documentation.md
├── 5_AI_ML_Integration/
│   ├── AI_ML_Integration.md
├── 6_System_Testing/
│   ├── Testing_Documentation.md
├── 7_Deployment/
│   ├── Deployment_Guide.md
├── 8_Project_Documentation/
│   ├── Project_Report.md ← This document
│   ├── backend/
│   │   ├── main.py
│   │   └── requirements.txt
│   ├── frontend/
│   │   ├── index.html
│   │   ├── static/
│   │   │   ├── style.css
│   │   └── app.js
```